

Time Report System — Production Sprint Plan

2 Sprints × 3 Days = 6 Days | 4 Developers | 24 Dev-Days Total

Team Roles

Dev	Role	Primary Focus
Dev 1	Backend Lead	APIs, business logic, database, validations
Dev 2	Backend	APIs, auth, middleware, export, notifications
Dev 3	Frontend Lead	Core user pages, reporting forms, monthly view
Dev 4	Frontend	Admin pages, layout, responsive, UI states

Daily Schedule Overview

Day	Sprint	Dev 1 (BE)	Dev 2 (BE)	Dev 3 (FE)	Dev 4 (FE)
D1	S1	Docker + DB schema + migrations	Auth API + JWT + middleware	Project scaffold + routing + layout shell	Design system components (Button, Input, Modal, Table, Toast, etc.)
D2	S1	Time entries API + overlap + validation	Users + Clients + Projects + Tasks + Assignments CRUD APIs	Login + change password + daily report page + form	Admin CRUD pages (users, clients, projects, tasks, assignments)
D3	S1	Absences API + file upload + monthly quota calc	Weekly submission + timer API + dropdown data endpoint	Monthly view + calendar + existing entries list + timer UI	Absence page + document upload + report history page
D4	S2	Approval + rejection API +	Month lock API + audit log	Weekly submit bar + approval/rejection UI	Month lock page + audit log viewer +

Day	Sprint	Dev 1 (BE)	Dev 2 (BE)	Dev 3 (FE)	Dev 4 (FE)
		admin report editing	middleware + audit log API	(admin) + rejection view (user)	admin dashboard
D5	S2	Export API (Excel + PDF) + notification triggers	Holiday calendar API + settings API + email sending	Dashboard FE + export page + notification bell + holiday settings FE	Mobile responsive pass (all screens) + UI states (loading, error, empty, etc.)
D6	S2	Integration testing + API fixes + Swagger docs	Permission tests + lifecycle tests + bug fixes	E2E user flow testing + bug fixes + polish	E2E admin flow testing + bug fixes + README

SPRINT 1: FOUNDATION + CORE (Days 1-3)

EPIC 1: Infrastructure & Authentication

Goal: Docker runs, DB ready, auth works end-to-end, app shell with routing

Story 1.1: Project Setup & Docker Environment

Assigned: Dev 1 (Day 1 morning) + Dev 3 (Day 1 morning) Points: 3

#	Task	Subtask	Owner
1.1.1	Initialize monorepo	Create root with docker-compose.yml, .gitignore, .env.example	Dev 1
1.1.2	Server scaffold	Create server/ with Express, folder structure (modules/, middleware/, database/, utils/), package.json	Dev 1
1.1.3	Client scaffold	Create client/ with Vite + React, folder structure (features/, components/, api/, hooks/, stores/, utils/), package.json	Dev 3
1.1.4	Docker Compose	3 services: client (port 3000), server (port 5000), postgres (port 5432). Volume mounts for hot reload. Health checks.	Dev 1
1.1.5	CI Pipeline	.github/workflows/ci.yml — install, lint, test, build. Branch protection on main.	Dev 1
1.1.6	Shared tooling	ESLint + Prettier for both. Testing framework setup (Jest or Vitest). Path aliases.	Dev 1

Acceptance: `docker-compose up` starts all 3 services. Client at :3000, Server at :5000/api.

Story 1.2: Database Schema & Migrations

Assigned: Dev 1 (Day 1) **Points:** 5

#	Task	Subtask	Owner
1.2.1	Migration system	Install Knex.js, configure connection, create runner script, add <code>npm run migrate</code> command	Dev 1
1.2.2	Core tables	<code>users</code> (id, email, password_hash, full_name, role enum, is_active, must_change_password, created_at, updated_at)	Dev 1
		<code>clients</code> (id, name, contact_info, is_active, created_at, updated_at)	
		<code>projects</code> (id, client_id FK, name, is_active, created_at, updated_at)	
		<code>tasks</code> (id, project_id FK, name, status enum open/closed, created_at, updated_at)	
		<code>user_task_assignments</code> (id, user_id FK, task_id FK, is_active, unique on active user+task, created_at, updated_at)	
1.2.3	Reporting tables	<code>time_entries</code> (id, user_id FK, date, start_time, end_time, duration_minutes, client_id FK, project_id FK, task_id FK, location enum, description, status enum, version int, created_at, updated_at)	Dev 1
		<code>absence_entries</code> (id, user_id FK, type enum, start_date, end_date, is_partial, notes, status enum, created_at, updated_at)	
		<code>attachments</code> (id, absence_id FK, file_name, file_path, mime_type, size_bytes, uploaded_by FK, created_at)	
1.2.4	Lifecycle tables	<code>weekly_submissions</code> (id, user_id FK, week_start_date, status enum, submitted_at, reviewed_by FK, reviewed_at, rejection_reason)	Dev 1
		<code>month_locks</code> (id, year, month, locked_by FK, locked_at, unlocked_by, unlocked_at, is_locked)	
1.2.5	System tables	<code>audit_logs</code> (id, actor_user_id, target_entity_type, target_entity_id, action, old_value jsonb, new_value jsonb, reason, timestamp, ip_address)	Dev 1
		<code>system_settings</code> (id, key unique, value jsonb, updated_by FK, updated_at)	
		<code>holiday_calendar</code> (id, date unique, name, type, created_by FK, created_at)	
		<code>active_timers</code> (id, user_id FK unique, start_time, created_at)	

#	Task	Subtask	Owner
		<code>permission_flags</code> (id, user_id FK, flag_name, scoped_project_ids jsonb, granted_by FK, created_at)	
		<code>notifications</code> (id, user_id FK, type, title, message, entity_type, entity_id, is_read, created_at)	
1.2.6	Indexes	time_entries(user_id, date), time_entries(user_id, date, start_time, end_time), weekly_submissions(user_id, week_start_date), audit_logs(target_entity_type, timestamp), absence_entries(user_id, start_date)	Dev 1
1.2.7	Seed data	Default admin user + test data (5 users, 3 clients, 5 projects, 10 tasks, assignments)	Dev 1

Acceptance: All migrations run and rollback cleanly. Seed data creates valid dataset.

Story 1.3: Authentication Backend

Assigned: Dev 2 (Day 1) **Points:** 5

#	Task	Subtask	Owner
1.3.1	Auth routes	POST /auth/login, /auth/logout, /auth/refresh, /auth/change-password	Dev 2
1.3.2	Login logic	Email lookup → bcrypt compare → generate access token (15 min) + refresh token (7 days) → return both	Dev 2
1.3.3	Token management	Refresh token rotation (new on each use, old invalidated). Store refresh tokens in DB.	Dev 2
1.3.4	Password service	bcrypt hash (cost 12), validate policy (8+ chars, upper, lower, digit, special), change-password endpoint (verify old, validate new, clear must_change_password)	Dev 2
1.3.5	Admin reset password	POST /users/:id/reset-password — generate temp password, set must_change_password=true, audit log	Dev 2
1.3.6	Account lockout	Track failed attempts. Lock 15 min after 5 consecutive failures. Return 423 Locked.	Dev 2
1.3.7	Auth middleware	Verify JWT from header, extract user to req.user, handle expired/invalid/missing.	Dev 2
1.3.8	RBAC middleware	Check user.role and permission flags. Middleware factory: <pre>requireRole('admin'), requirePermission('canAssignProjectTasks')</pre>	Dev 2
1.3.9	Session kill	On user deactivation: invalidate all refresh tokens. On password change: invalidate other sessions.	Dev 2
1.3.10	Rate limiting	10 login/min per IP, 100 req/min per user	Dev 2
1.3.11	Tests	Login success, wrong password 401, lockout 423, refresh rotation, forced password change, admin reset, deactivated user rejected	Dev 2

Acceptance: Login returns tokens. Refresh rotates. Lockout works. Forced password change works.

Story 1.4: Authentication Frontend + App Shell

Assigned: Dev 3 (Day 1) + Dev 4 (Day 1) **Points:** 5

#	Task	Subtask	Owner
1.4.1	App shell & routing	App.jsx with React Router. ProtectedRoute component. Route config for all pages (can be empty placeholders).	Dev 3
1.4.2	Layout components	AppShell.jsx (sidebar + header + content area). Sidebar.jsx with nav items by role. Header.jsx with user name + logout. MobileNav.jsx (hamburger menu). RTL direction on html.	Dev 4
1.4.3	API client	Axios instance with base URL, auth interceptor (attach token), 401 interceptor (auto-refresh), error handler.	Dev 3
1.4.4	Auth context	AuthContext.jsx: login(), logout(), refresh(), user state, isAdmin flag, isAuthenticated flag. Persist token per Remember Me.	Dev 3
1.4.5	Login page	LoginPage.jsx: email, password, Remember Me checkbox, login button. Hebrew labels, RTL. Error display. Lockout message.	Dev 3
1.4.6	Change password page	ChangePasswordPage.jsx: current password, new password, confirm. Strength indicator. Redirect to home on success.	Dev 3
1.4.7	Inactivity timeout	30 min idle → warning dialog → auto-logout. Reset timer on any interaction.	Dev 3

Acceptance: Login works. Forced password change redirects. Sidebar shows correct nav by role. Auto-logout on idle.

Story 1.5: Design System Components

Assigned: Dev 4 (Day 1) **Points:** 3

#	Task	Subtask	Owner
1.5.1	Button	Primary, secondary, danger, disabled states. Loading spinner variant.	Dev 4
1.5.2	Input + Select	Text input, password input, textarea. Select/dropdown with search. RTL support. Error state (red border + message).	Dev 4
1.5.3	Modal + ConfirmDialog	Modal wrapper. ConfirmDialog with title, message, confirm/cancel buttons, danger variant for destructive actions.	Dev 4
1.5.4	Table	Sortable columns, pagination, empty state, loading skeleton, mobile card variant.	Dev 4
1.5.5	Toast	Success/error/warning. Auto-dismiss. Stack multiple.	Dev 4
1.5.6	DatePicker + TimePicker	Hebrew locale, RTL. Date range support for absences.	Dev 4
1.5.7	FileUpload	Drag-and-drop + click. File type/size validation. Progress bar.	Dev 4
1.5.8	Utility components	Spinner, EmptyState (illustration + text + CTA), ErrorState (retry button), Badge (status colors).	Dev 4

Acceptance: All components render correctly in RTL. States work (loading, error, disabled).
Used across all pages.

EPIC 2: Entity Management (Admin CRUD)

Goal: Admin can manage all entities. Users can be assigned to tasks.

Story 2.1: All Entity APIs (Backend)

Assigned: Dev 2 (Day 2) **Points:** 8

#	Task	Subtask	Owner
2.1.1	Users CRUD	GET /users (list + filters), GET /users/:id, POST /users (create with initial password, must_change_password=true), PUT /users/:id, DELETE /users/:id (soft delete → is_active=false, kill sessions). GET /users/me. Validate email unique.	Dev 2
2.1.2	Permission flags	POST /users/:id/permissions (grant canAssignProjectTasks with scoped_project_ids), DELETE /users/:id/permissions/:id (revoke), GET /users/:id/permissions	Dev 2
2.1.3	Clients CRUD	GET /clients (Admin: all; User: only assigned), POST /clients (name, contact_info, default active), PUT, DELETE (soft delete with warning if has projects)	Dev 2
2.1.4	Projects CRUD	GET /projects (filter by client, role-scoped), POST (client_id, name, warn duplicate), PUT, DELETE (soft delete with warning if has tasks)	Dev 2
2.1.5	Tasks CRUD	GET /tasks (filter by project, status), POST (project_id, name, default open), PUT, DELETE (set closed)	Dev 2
2.1.6	Assignments	GET /assignments (role-scoped), POST (user_id, task_id, validate no active duplicate, check canAssignProjectTasks scope), PUT (toggle active), DELETE (soft delete)	Dev 2
2.1.7	Audit logging	All CRUD mutations auto-logged via middleware: actor, entity, action, old/new value, timestamp	Dev 2
2.1.8	Tests	Create, read, update, delete for each entity. Permission checks. Soft delete behavior.	Dev 2

Acceptance: All 6 entity APIs work with correct permissions. Soft delete preserves history. Audit logged.

Story 2.2: Admin CRUD Pages (Frontend)

Assigned: Dev 4 (Day 2) **Points:** 8

#	Task	Subtask	Owner
2.2.1	User list + form	Table: name, email, role badge, status badge, actions (edit, deactivate, reset password). Modal form: name, email, password (create), role dropdown, active toggle. ResetPasswordDialog. Permission flag section.	Dev 4
2.2.2	Client list + form	Table: name, contact, status, project count. Modal form: name, contact info. Archive confirmation with warning.	Dev 4
2.2.3	Project list + form	Table: client, name, status, task count. Modal form: client dropdown, name. Filter by client. "Add task" inline button.	Dev 4
2.2.4	Task list + form	Table: project, name, status badge. Modal form: project dropdown, name. Status toggle.	Dev 4
2.2.5	Assignment page	Matrix or table view. User × Task grid with checkboxes. Filter by project, user. Scope indicator for User+flag.	Dev 4
2.2.6	Shared patterns	All pages: search bar, status filter, pagination, empty state, loading skeleton, delete confirmation with impact warning, success/error toasts	Dev 4

Acceptance: Admin can create, edit, archive all entities. Assignments work. All soft delete with confirmation.

EPIC 3: Time Reporting (Core User Flow)

Goal: User can report daily hours, see monthly view, use timer

Story 3.1: Time Entries API

Assigned: Dev 1 (Day 2–3) **Points:** 8

#	Task	Subtask	Owner
3.1.1	CRUD endpoints	GET /time-entries (filter by user, date, week, month), GET /:id, POST (create with auto duration calc), PUT (version check for optimistic locking), DELETE (soft)	Dev 1
3.1.2	Overlap detection	overlapDetector.js — query existing entries for same user+date, check if new start/end overlaps any. Handle cross-midnight spans. Allow adjacent (end=start).	Dev 1
3.1.3	Cross-midnight calc	durationCalculator.js — if end < start, assume cross-midnight, calculate correctly. Entry belongs to start date.	Dev 1
3.1.4	Validation	All fields required. End > start (except cross-midnight). No future dates. Task must be active and assigned to user. Project/client must be active. Month not locked. Entry not submitted (unless rejected).	Dev 1
3.1.5	Dropdown data	GET /time-entries/dropdown-data — return user's client→project→task tree from assignments. Support sort param: alphabetical or frequency.	Dev 1
3.1.6	Daily summary	GET /time-entries/daily-summary?date= — total hours, remaining vs standard, entry count, status per entry	Dev 1
3.1.7	Monthly summary	GET /time-entries/monthly-summary?year=&month= — per-day: total hours, status (full/missing/irregular), entry count. Include absences.	Dev 1
3.1.8	Monthly quota	GET /time-entries/monthly-quota?year=&month= — working days, holidays, absences, quota, reported, remaining	Dev 1
3.1.9	quotaCalculator.js	(working days × 9h) – holidays – full absences(×9h) – partial absences(×4.5h) ± admin exceptions	Dev 1
3.1.10	Tests	Overlap blocked, adjacent allowed, cross-midnight correct, future blocked, locked month blocked, quota calc, dropdown tree correct	Dev 1

Acceptance: All validations enforced. Overlap blocked across projects. Cross-midnight works. Quota accurate.

Assigned: Dev 3 (Day 2) Points: 8

#	Task	Subtask	Owner
3.2.1	DailyReportPage.jsx	Default home screen after login. Date picker (default today). Daily standard display (9h). Progress bar (reported/standard).	Dev 3
3.2.2	ExistingEntriesList.jsx	List all entries for selected date. Each row: time range, client/project/task, description, edit/delete buttons. Highlight rejected entries with reason. Disabled if submitted/locked.	Dev 3
3.2.3	ReportForm.jsx	Location dropdown (Office/Client/Home). Start time + end time (default now). Auto-calc duration. Cascading dropdowns: client→project→task. Auto-select if single option. Description textarea.	Dev 3
3.2.4	Cascading dropdowns	Fetch dropdown tree from API. On client select → filter projects. On project select → filter tasks. Clear downstream on parent change. Sort toggle (alpha/frequency) persisted.	Dev 3
3.2.5	Save/Cancel flow	Save: POST entry, success toast, reset form to blank for next entry. Cancel: discard form. Validation: all fields required, field-level errors.	Dev 3
3.2.6	Warnings	Under-standard banner: "You have 3 hours remaining today". Over-standard banner. Overtime message if over monthly quota.	Dev 3

Acceptance: Full daily reporting works. Cascading dropdowns correct. Save resets form. Warnings display.

Story 3.3: Monthly View + Report History

Assigned: Dev 3 (Day 3) **Points:** 5

#	Task	Subtask	Owner
3.3.1	MonthlyViewPage.jsx	Month/year navigator. Calendar grid with day cells.	Dev 3
3.3.2	Day cells	Color-coded: green (מלא/full), red (חסר/missing), yellow (חרוג/irregular). Click to expand entries.	Dev 3
3.3.3	QuotaProgressBar.jsx	X / Y hours bar. Color changes at 80% and 100%. Shows remaining.	Dev 3
3.3.4	Entry detail list	Below calendar: entries for selected day. Date, time range, client/project/task, description. Click to edit (if not locked).	Dev 3
3.3.5	Locked month banner	"January 2026 is locked — read only" with lock icon. All actions disabled.	Dev 3
3.3.6	ReportHistoryPage.jsx	Separate page. Month picker to browse past months. Flat table of all entries (not calendar). Includes locked months (read-only). Filter by client/project/task.	Dev 3

Acceptance: Calendar shows correct day statuses. Quota bar accurate. History browses past months.

Story 3.4: Timer

Assigned: Dev 2 (Day 3 backend) + Dev 3 (Day 3 frontend, if time permits on Day 4) **Points:** 3

#	Task	Subtask	Owner
3.4.1	Timer API	POST /timer/start (validate no active timer), POST /timer/stop (return start/end), GET /timer/status. Server-side persistence in active_timers table.	Dev 2
3.4.2	Auto-stop	Background job: stop timers > 12 hours.	Dev 2
3.4.3	TimerButton.jsx	Start/stop button on daily page. Running state: elapsed counter, pulsing dot.	Dev 3
3.4.4	TimerCompletionDialog.jsx	On stop: dialog with pre-filled times, empty location/client/project/task/description fields. Save creates time entry.	Dev 3
3.4.5	Timer guards	Block weekly submission if timer running. Warning at 10h.	Dev 2

Acceptance: Timer starts/stops. One at a time. Persists across refresh. Auto-stop at 12h. Blocks submit.

EPIC 4: Absences

Goal: User can report absences with document upload

Story 4.1: Absence API

Assigned: Dev 1 (Day 3) **Points:** 5

#	Task	Subtask	Owner
4.1.1	CRUD endpoints	GET /absences (filter by user, month, type), POST (type, dates, is_partial, notes), PUT (pre-submit only), DELETE (pre-submit only)	Dev 1
4.1.2	Date logic	Auto-exclude Friday/Saturday from range. Calculate day count. Partial = 0.5 days.	Dev 1
4.1.3	Document upload	POST /absences/:id/documents — validate type (PDF/JPG/JPEG/PNG/DOC/DOCX), max 10MB, server-side MIME check. Store file. DELETE /absences/:id/documents/:id. Allow upload even after submission/lock (audit logged).	Dev 1
4.1.4	Quota integration	Absence reduces monthly quota: full day = -9h, partial = -4.5h. Feed into quotaCalculator.	Dev 1
4.1.5	Validations	Sick requires document (warn if missing). Reserve requires document. Other has optional notes. Partial requires work entries for remaining hours. Future absences allowed.	Dev 1
4.1.6	Tests	Fri/Sat excluded, quota reduced, file type/size validated, post-lock upload audit logged	Dev 1

Acceptance: All 4 absence types work. Fri/Sat excluded. Document upload validated. Quota updated.

Story 4.2: Absence Frontend

Assigned: Dev 4 (Day 3) **Points:** 3

#	Task	Subtask	Owner
4.2.1	AbsenceReportPage.jsx	Form + list of existing absences	Dev 4
4.2.2	AbsenceForm.jsx	Type dropdown (4 types), date range picker (Fri/Sat greyed out), partial checkbox, notes textarea	Dev 4
4.2.3	DocumentUpload.jsx	Drag-drop or file picker. Type/size indicator. Download link for existing docs.	Dev 4
4.2.4	AbsenceList.jsx	Table: type, dates, status, document indicator, edit/delete. Disabled if submitted/locked.	Dev 4

Acceptance: All absence types reportable. Date range works. Document upload works.

SPRINT 2: LIFECYCLE + ADMIN OPS + POLISH (Days 4-6)

EPIC 5: Reporting Lifecycle

■ **Goal:** Weekly submit → admin review → approve/reject → month lock

Story 5.1: Weekly Submission API

Assigned: Dev 1 (Day 4) **Points:** 5

#	Task	Subtask	Owner
5.1.1	Submit endpoint	POST /weekly-submissions/:id/submit — validate all working days fully allocated (hours + absences = standard), no running timer, no validation errors. Set status=submitted, lock entries.	Dev 1
5.1.2	Weekly data	GET /weekly-submissions (list by user, month). GET /:id with linked entries.	Dev 1
5.1.3	Auto-flag Missing	Cron: Sunday 23:59 — flag unsubmitted weeks as Missing.	Dev 1
5.1.4	Resubmission	After rejection: entries return to draft. User edits and re-submits same week.	Dev 1
5.1.5	Tests	Cannot submit under-allocated, cannot submit with timer, submission locks entries, Missing auto-flagged	Dev 1

Story 5.2: Approval & Rejection API

Assigned: Dev 1 (Day 4) **Points:** 5

#	Task	Subtask	Owner
5.2.1	Approval endpoint	POST /weekly-submissions/:id/approve — set all entries status=approved	Dev 1
5.2.2	Rejection (per entry)	POST /time-entries/:id/reject — mandatory reason, set status=rejected, entry returns to draft	Dev 1
5.2.3	Rejection (per week)	POST /weekly-submissions/:id/reject — reject entire week with reason	Dev 1
5.2.4	Pending list	GET /approvals/pending — all submitted weeks across users	Dev 1
5.2.5	Notifications	Create notification on rejection. Trigger email to user.	Dev 1
5.2.6	Admin editing	Admin can PUT any user's time entry. Change is audit logged with old/new values. Entry shows "Modified by Admin" flag.	Dev 1
5.2.7	Tests	Approve updates statuses, reject requires reason, rejected entry editable, resubmit works	Dev 1

Story 5.3: Lifecycle Frontend (User Side)

Assigned: Dev 3 (Day 4) **Points:** 5

#	Task	Subtask	Owner
5.3.1	WeeklySubmitBar.jsx	Shows on daily page + monthly view. Week status badge. Submit button (enabled only when ready). Disabled tooltip explanation.	Dev 3
5.3.2	Submit confirmation	Dialog: "Submit week of May 1-5? You won't be able to edit after submission."	Dev 3
5.3.3	Missing week alert	Banner: "You haven't submitted last week's report" with action button	Dev 3
5.3.4	Rejection view	Rejected entries: red border, rejection reason text, "Edit & Resubmit" button	Dev 3
5.3.5	Admin edit indicator	Badge: "Modified by Admin on [date]" on entries edited by admin. Click to see changes.	Dev 3
5.3.6	Timer submission guard	If timer running: "Stop your timer before submitting" with link to stop	Dev 3

Story 5.4: Lifecycle Frontend (Admin Side)

Assigned: Dev 3 (Day 4) **Points:** 5

#	Task	Subtask	Owner
5.4.1	ReportReviewPage.jsx	List of submitted weeks. Filter by user, status, week.	Dev 3
5.4.2	WeeklyReviewCard.jsx	Expandable card: user name, week dates, entry count, total hours. Expand shows all entries.	Dev 3
5.4.3	Entry-level actions	Per entry: approve (check) or reject (X) button.	Dev 3
5.4.4	RejectReasonDialog.jsx	Mandatory text input. Cannot save empty.	Dev 3
5.4.5	Bulk approve	"Approve Entire Week" button at card level.	Dev 3
5.4.6	Status colors	Submitted=blue, Approved=green, Rejected=red, Draft=grey	Dev 3

Assigned: Dev 2 (Day 4 backend) + Dev 4 (Day 4 frontend) **Points:** 5

#	Task	Subtask	Owner
5.5.1	Lock API	POST /months/:year/:month/lock — set is_locked=true, record actor+timestamp. POST /months/:year/:month/unlock — require reason. GET status. GET /months list.	Dev 2
5.5.2	Lock enforcement	All time entry/absence mutations check month lock before proceeding. Return 403 with "Month is locked" message.	Dev 2
5.5.3	MonthLockPage.jsx	Month grid with status (open/locked). Lock button + confirm dialog. Unlock button + reason dialog. Warning if unapproved weeks exist.	Dev 4
5.5.4	User-side indicators	Locked month: banner on monthly view, disable all actions, grey out forms with tooltip.	Dev 4
5.5.5	Tests	Lock blocks edits, unlock requires reason, audit logged	Dev 2

EPIC 6: Admin Operations

Goal: Dashboard, audit log, export, notifications, settings

Story 6.1: Audit Log

Assigned: Dev 2 (Day 4 backend) + Dev 4 (Day 4 frontend) **Points:** 5

#	Task	Subtask	Owner
6.1.1	Audit middleware	auditLog.middleware.js — auto-log all POST/PUT/DELETE. Capture actor, entity, action, old/new values, timestamp, IP.	Dev 2
6.1.2	Audit API	GET /audit-logs — Admin only. Filters: entity_type, entity_id, actor_id, action, date range. Pagination.	Dev 2
6.1.3	AuditLogPage.jsx	Table: timestamp, actor, action, entity, details. Expandable row for old/new diff. Filters. Pagination.	Dev 4

Assigned: Dev 1 (Day 5 backend) + Dev 3 (Day 5 frontend) **Points:** 5

#	Task	Subtask	Owner
6.2.1	Dashboard API	GET /admin/dashboard?year=&month= — matrix: users × weeks, cell = status. Summary counts.	Dev 1
6.2.2	AdminDashboard.jsx	Default admin home. Summary cards (total, submitted, missing, approved).	Dev 3
6.2.3	SubmissionStatusTable.jsx	Color-coded matrix. Click cell → navigate to review page.	Dev 3
6.2.4	Month selector	Dropdown or arrows to navigate months.	Dev 3

Story 6.3: Export

Assigned: Dev 2 (Day 5) **Points:** 5

#	Task	Subtask	Owner
6.3.1	Export API	GET /exports?month=&year=&user_id=&client_id=&project_id=&format=xlsx	pdf. Admin only.
6.3.2	Excel generation	Using exceljs: all entry data, absences, approval status, correction history.	Dev 2
6.3.3	PDF generation	Using pdfkit or puppeteer: same data formatted for print.	Dev 2
6.3.4	Watermark	If month not approved: "NOT APPROVED" watermark on every page.	Dev 2
6.3.5	Payroll lock	If payroll flag: auto-lock month after export.	Dev 2
6.3.6	Export metadata	Timestamp + generating admin ID embedded in file. Audit logged.	Dev 2
6.3.7	ExportPage.jsx	Filter controls, preview summary, format toggle, payroll checkbox, download button. Warning if unapproved.	Dev 3

Story 6.4: Notifications & Settings

Assigned: Dev 2 (Day 5 backend) + Dev 3 (Day 5 frontend) **Points:** 5

#	Task	Subtask	Owner
6.4.1	Notifications API	POST (internal, created by events), GET /notifications (user's unread), PUT /:id/read.	Dev 2
6.4.2	Email triggers	Rejection email, Thursday missing-report reminder (cron). Use nodemailer.	Dev 2
6.4.3	Holiday calendar API	GET /settings/holidays, POST (Admin), DELETE (Admin). Feeds into quota calc.	Dev 2
6.4.4	System settings API	GET /settings, PUT /settings (Admin). Daily standard hours config.	Dev 2
6.4.5	NotificationBell.jsx	Header icon. Unread count badge. Dropdown list. Click → navigate.	Dev 3
6.4.6	HolidayCalendarPage.jsx	Calendar view. Add/remove holidays.	Dev 3
6.4.7	SystemSettingsPage.jsx	Configure daily standard hours.	Dev 3

EPIC 7: Quality & Launch

■ **Goal:** Mobile responsive, UI states, testing, documentation, deploy

Story 7.1: Mobile Responsive

Assigned: Dev 4 (Day 5) **Points:** 5

#	Task	Subtask	Owner
7.1.1	User pages mobile	Daily report form: stack fields vertically. Monthly calendar: scrollable or list view. Report history: card layout. Absence form: full-width fields.	Dev 4
7.1.2	Admin pages mobile	Admin tables: switch to card layout. Dashboard matrix: horizontal scroll or stacked. Approval cards: full-width.	Dev 4
7.1.3	Navigation mobile	MobileNav.jsx: bottom tab bar or hamburger. Touch-friendly tap targets (44px min).	Dev 4
7.1.4	Modal mobile	Full-screen modals on mobile instead of floating. Swipe-to-close.	Dev 4

Story 7.2: UI States (All Screens)

Assigned: Dev 4 (Day 5) **Points:** 3

#	Task	Subtask	Owner
7.2.1	Loading states	Add Spinner/skeleton to all data-fetching pages	Dev 4
7.2.2	Empty states	Add EmptyState component to all list pages (illustration + message + CTA)	Dev 4
7.2.3	Error states	Add ErrorState with retry button on all pages	Dev 4
7.2.4	Offline state	Banner: "You are offline" when no network detected	Dev 4
7.2.5	Unsaved changes	useUnsavedChanges hook: warn on navigation away from dirty forms	Dev 4
7.2.6	Disabled tooltips	All disabled buttons have tooltip explaining why (e.g., "Month is locked")	Dev 4

Story 7.3: Testing & Documentation (Day 6 — ALL DEVS)

Assigned: All devs **Points:** 8

#	Task	Subtask	Owner
7.3.1	Unit tests	Validation logic, quota calc, overlap detection, duration calc, cross-midnight	Dev 1
7.3.2	Integration tests	All API endpoints with real DB. Auth flow. CRUD operations.	Dev 2
7.3.3	Permission tests	Every cell in permission matrix. Both allowed and denied.	Dev 2
7.3.4	Lifecycle tests	Draft→Submit→Approve→Lock. Rejection→Edit→Resubmit. Missing auto-flag.	Dev 1
7.3.5	E2E user flow	Login → report → submit week → see approval → see locked month	Dev 3
7.3.6	E2E admin flow	Login → review → approve → reject → lock month → export	Dev 3
7.3.7	Swagger docs	Document all endpoints with request/response schemas	Dev 2
7.3.8	README	Installation instructions, Docker setup, environment variables, run commands	Dev 4
7.3.9	CD deployment	Final deploy to production hosting. Verify all features live.	Dev 1
7.3.10	Bug fixes	Fix any issues found during testing	All

Acceptance: 60%+ test coverage. Swagger accessible. README complete. Production deployed.

Summary Stats

Metric	Count
Epics	7
Stories	19
Tasks	~95
Subtasks	~280

Metric	Count
API endpoints	~45
Database tables	16
Frontend pages	~25
Dev-days	24 (4 devs × 6 days)

Risk Mitigation

Risk	Mitigation
Day 1 Docker issues eat time	Dev 1 has Docker ready by lunch. If delayed, others work on local Node/Postgres.
Backend blocks frontend	Frontend uses mock data / JSON fixtures on Day 1-2 while APIs are built. Switch to real APIs as they become available.
Testing takes longer than Day 6	Write tests alongside features (not just Day 6). Each backend story includes tests. Day 6 is for integration + E2E.
Scope creep	Refer ONLY to this plan. Any feature not listed here is post-launch.
Design gaps (Figma incomplete)	Dev 4 makes pragmatic UI decisions based on master spec. Polish in post-launch sprint.

Definition of Done (Every Story)

- ☐ Code works in Docker environment
- ☐ API endpoints documented in Swagger
- ☐ Unit tests pass (60%+ coverage for the module)
- ☐ Responsive on mobile and desktop
- ☐ RTL Hebrew layout correct
- ☐ Audit logged (if mutation)
- ☐ Permission-checked (if restricted)
- ☐ Code reviewed by at least 1 other dev